Linux Signal-Based Process Controller

2520090xxx – Hrushika | 2520090042 Chakrika | 2520080012 - Saranya

Operating Systems project | C + Python (Tkinter) | POSIX signals

What the project does

A central controller (sigctld) creates real worker processes with fork() and execv(), controls them with real POSIX signals, and recovers failed ones automatically. A dashboard shows only verified state.

Real control

Pause = SIGSTOP
Resume = SIGCONT
Stop = SIGTERM, then SIGKILL after a grace period. Nothing is simulated.

Verified state

Paused only after waitpid(WUNTRACED) reports it. Exited only after the status is reaped. /proc gives CPU and memory.

Self-healing

A watchdog restarts crashed workers with a restart limit and exponential backoff, and logs every decision.

Architecture

Clients

Tkinter dashboard
sigctl command-line client
Address workers by registry ID, never by PID.

Controller: sigctld

Event loop, registry, watchdog, JSON-lines logger, IPC server.
Single instance (flock).

Workers: sigworker

primes, hasher, sorter, plus test profiles crashy, flaky, sleepy.

Unix domain socket (SO_PEERCRED)

fork() + execv(), signals, pipe replies

Signals used

Action	Signal	Confirmed by
Pause	SIGSTOP	waitpid(WUNTRACED)
Resume	SIGCONT	WCONTINUED or handshake
Status report	SIGUSR1	reply through worker pipe
Diagnostic snapshot	SIGUSR2	reply through worker pipe
Graceful stop	SIGTERM (then SIGKILL)	waitpid() exit status
Child ended	SIGCHLD	reaped, no zombies

Handlers only set flags and write to a self-pipe (async-signal-safe). Workers block signals and read them with sigtimedwait().

Intelligent watchdog

It tells failures apart

Intentional stop vs crash, crash vs normal exit, start failure vs runtime failure, slow worker vs dead worker.

Restart policy

max_restarts = 3 consecutive failures. Delay starts at 1 s and doubles to a 16 s cap. Count resets after 30 s of stable uptime.

Health checks

Every 500 ms via /proc. A status request unanswered within 2 s marks the worker unresponsive.

Fault injection

FAULT <id> crash | segv | hang | exit makes a worker fail on purpose for testing.

Safety and robustness

- Signals only its own unreaped children; parent PID checked in /proc before every kill()
- Each worker in its own process group; PR_SET_PDEATHSIG stops orphans if the controller dies
- Single-instance flock, stale-socket recovery, same-user check, refuses to run as root
- SIGHUP reloads the configuration; every event persisted to a JSON-lines log

Testing and how to run

Testing

72 unit checks
21 integration tests on real processes (14 required scenarios + 7 edge cases)
GUI smoke test
All pass under AddressSanitizer and UndefinedBehaviorSanitizer

Run it (Linux / WSL2)

./scripts/setup_ubuntu.sh
make
make test
./scripts/start.sh

CLI: ./build/sigctl SPAWN primes, then LIST, PAUSE 1, RESUME 1, STOP 1